Principiile SOLID

- Single Responsibility Principle
- Open Closed Principle
- Liskov Substitution Principle
- Interface Segregation Principle
- Dependency Inversion Principle

Single Responsability Principle

- Fiecare clasa trebuie sa aiba o singura responsabilitate
- Orice modul sau clasa trebuie sa incapsuleze o singura functionalitate
- O clasa sau functie nu trebuie sa rezolve sau sa trateze mai mult decat un singur scop deoarece ar rezulta cuplarea intre cele doua functionalitati

Open Closed Principle

- Cand o componenta software este implementata si utilizata, orice modificare ulterioara a acelei componente poate duce la erori sau comportament nedefinit
- Componentele software, odata implementate, nu ar trebui sa fie modificate, ci sa-si extinda functionalitatea prin alte mijloace

Liskov Substitution Principle

- Orice clasa derivata poate inlocui clasa de baza si codul sa functioneze corect in continuare
- Acest principiu presupune ca intr-un program daca S este subtip al lui T, atunci obiectele de tip T pot fi inlocuite cu obiecte de tip S
- Noile clase extinse din parinte trebuie sa fie capabile sa inlocuiasca clasa de baza in toate functiile sale fara a fi nevoie sa aducem modificari nicaieri

Interface Segregation Principle

- In loc sa avem o interfata cu foarte multe metode, din care doar cateva sunt folosite, mai bine avem mai multe interfete cu mai putine metode si le folosim pe cele necesare
- Acest principiu se realizeaza prin "spargerea" interfetelor in module mai mici pe care clasele au optiunea de a le implementa sau nu
- Acest mod va ajuta si la ideea de decuplare a functionalitatilor atunci cand se doreste refactorizarea produsului

Dependency Inversion Principle

- Daca o clasa are o dependenta, aceasta dependenta ar trebui sa fie o interfata, nu o clasa concreta

Concepte POO de baza

Clasa : reprezinta un tip de data
Obiect : este o instanta a unei clase
Mesaj : este folosit de obiecte pentru a comunica, un mesaj este un apel de metoda
Incapsularea : separarea reprezentarii de interfata
Mostenirea : reutilizarea codului, definirea unei ierarhii de obiecte
Polimorfismul : capacitatea unei entitati de a reactiona diferit in functie de context

Sintaxa Java

Cuvinte cheie

abstract	double	int	super
boolean	else	interface	switch
break	extends	long	synchronized
byte	final	native	this
case	finally	new	throw
catch	float	package	throws
char	for	private	transient
class	(goto)	protected	try
(const)	if	public	void
continue	implements	return	volatile
default	import	short	while
do	instanceof		

Literali
"Hello World", 'J', 'a', 'v', 'a', 10, 010, 0xA, 0b11, 12.3, 12.3d, 12.3f, 123L, true, false, null

Separatori
( ) { } [ ] ; , .

Operatori
Acces, method call : ., [ ], ( )
Postfix : expr++, expr--
Other unary : ++expr, --expr, +, -, ~, !, new
Arithmetic : *, /, %
Aditive : +, -
Shift : <<, >>, >>>
Relational : <, >, <=, >=
Equality : ==, !=
Logical : &, ^, |, &&, ||
Ternary : condition ? (expr1) : (expr2)
Assigment : =, +=, -=, *=, /=, %=, ^=, |=, <<=, >>=, >>>=

Comentarii
Single line : //single line comment
Multiple lines : /*multi-line
comment*/
Javadoc comment : /** processed by javadoc */

Codificarea caracterelor - UNICODE
- Inlocuieste setul de caractere ASCII
- Un caracter se reprezinta pe 2 octeti
- 65536 caractere, **\uxxxx** (0x0041 = A)
- Compatibil ASCII, primele 256 de caractere fiind cele din ASCII

Tipuri de date
Primitive : Boolean, char(16), byte(8), short(16), int(32), long(64), float(32), double(64)
Referinta : clasa, interfata, [ ]
Observatii : void nu este tip in Java, este doar un cuvant cheie pentru cazurile in care dorim sa indicam ca ceea ce se returneaza este nimic

# Variabile si constante
byte a; //declarare
int valoare = 100; //declarare + initializare
final double PI = 3.142; //constanta
long numarElemente = 12345678L; //sufixul L indica literal de tip long
String bauturaMeaPreferata = "apa";

Constante
final int MAX = 100;
final int MAX2;
MAX2 = 100;
MAX2 = 150; //eroare

Var (tip de date inferat automat)
var str = "Java 10"
var list = new ArrayList<String>();

Instructiuni

`if (booleanExpr){`
`  // do something`
`}else { // else is optional`
`  // do something else`
`}`
	`for (Tip variable : collection) {  //Java 5.0`
	`  // loop body`
	`}`
	`for(Student s: studentsList){`
	`  System.out.println(s);`
	`}`

`while (booleanExpr) {`
`  // execute body`
`  // until booleanExpr becomes false`
`}`
 	
`do {`
`  // execute body (at least once)`
`  // until booleanExpr becomes false`
`}while (booleanExpr);`
 	
`for (int i=0; i < n; i++) {`
`  // execute loop body n times`
`}`
	`switch (Expr){`
	`  case Value1: instructions;`
	`  break;`
	`  case Value2: instructions;`
	`  break;`
	`  // ...`
	`  default: instructions;`
	`}`
	Expr poate fi:
	`  byte, short, int, char` (sau clase învelitoare)
	`  enum` types
	`  String (equals) (new` in Java 7)

Tablouri Unidimensionale

Declarare

tip nume_tablou[ ];
tip[ ] nume_tablou;

Creare

nume_tablou = **new** tip[dim]; //se aloca spatiul in memorie
//indexarea 0 ... dim-1

Referirea unui element

nume_tablou[index]

```
float[ ] vec;
vec = new float[10];
int[ ] sir = new int[3];
float tmp[ ];
tmp = vec;
```

Lenght

lenght returneaza dimensiunea (capacitatea) tabloului

```
int[ ] sir = new int[5];
int lung_sir = sir.lenght; //lung_sir = 5
```

Creare si initializare

int[ ] dx = {-1, 0, 1}

Tablouri N-dimensionale

Rectangulare

```
int[ ][ ] a;
a = new int[5][5];
a[0][0] = 2;
int x = a[2][2]; // x = 0
```

Nerectangulare

```
int[ ][ ] a = new int[3][ ];
for (int i = 0; i < 3; i++)
    a[i] = new int[i+1];
```

int x = a.lenght; //x = 3
int y = a[2].lenght; //x = 3

Declararea/definirea unei clase in Java

[**public**][**abstract**][**final**] class NumeClasa {
definitii si implementare date + metode
clase imbricate
}

Observatii

1. Daca clasa **NumeClasa** este declarata public, atunci clasa se salveaza intr-un fisier cu numele **NumeClasa.java**
2. Intr-un fisier **.java** pot fi definite mai multe clase, insa cel mult una poate fi **publica**
3. Diferente fata de C++ :
   - Nu sunt necesare 2 fisiere separate (.h, .cpp)
   - Metodele se implementeaza in locul declararii
   - La sfarsitul clasei nu se pune ;

Declararea datelor membru

[**public**][**abstract**][**final**] class NumeClasa {
[modificator_acces][**abstract**][**static**][**final**] Tip nume[=val_initiala];
}

Modificatorul de acces trebuie precizat pentru fiecare atribut si are urmatoarea semnificatie:

- **private** : membru accesibil doar clasei
- **protected** : membru accesibil doar clasei si subclaselor
- **public** : membru accesibil tuturor
- implicit : membru accesibil doar in interiorul pachetului (directorului)

Declararea datelor membru

```
//Persoana.java
public class Persoana{
    private String nume;
    private int varsta;
//...
}
```

```
//Punct.java
public class Punct{
    protected double x;
    protected double y;
//...
}
```

```
//Cerc.java
public class Cerc{
    double raza;
    Punct centru;
    static final double PI=3.14;
}
```

Declararea/definirea metodei constructor

Constructorul precizeaza o secventa de instructiuni care se vor executa imediat dupa crearea obiectului.

[...] class NumeClasa {
[modificator_acces] NumeClasa([lista_parametrilor_formali]) {
//secventa instructiuni
}
}

```
public class Complex{
    private double real, imag;
    public Complex(){
        real=0;
        imag=0;
    }
    …
```

Supraincarcarea constructorilor

```
public class Complex{
    private double real, imag;
    public Complex(){
        real=0;
        imag=0;
    }
    public Complex(double real){
        this.real=real;
        imag=0;
    }
    public Complex(double real, double imag){ //...
    }
    public Complex(Complex c){ //...
    }
}
```

Referinta la obiectul curent

this : putem referi atributele sau metodele la obiectul curent

```
public class Complex{
   private double real, imag;
   Complex(double real){
      this.real=real;
      this.imag=0;
   }
   Complex(double real, double imag){
      this.real=real;
      this.imag=imag;
   }
   public Complex suma (Complex c){
      // compute the sum
      return this;
   }
}
```

Apelul unui alt constructor

Un anumit constructor poate sa apeleze un alt constructor al aceleiasi clase. Apelul unui alt constructor trebuie sa fie prima instructiune din constructorul apelant.

```
public class Complex{
   private double real, imag;
   Complex(double real){
      this(real,0);
   }
   Complex(double real, double imag){
      this.real=real;
      this.imag=imag;
   }
   public Complex suma (Complex c){
      this(real+c.real,imag+c.imag);
      return this;
   }
}
```

Distrugerea obiectelor

In Java nu exista destructor, memoria este dealocata de garbage collector.

Transmiterea parametrilor

Prin valoare, se copiaza valoarea lor pe stiva.
Pentru parametri de tip obiect si pentru tablouri, aceasta valoare este o referinta.

Vector de obiecte

```
Student [] studentsList=new Student [4];
studentsList[0]=new Student("Andrei",12,3);
studentsList[1]=new Student("Aprogramatoarei",18,1);
studentsList[2]=new Student("Dan",32,3);
studentsList[3]=new Student("Ion",22,3);

for(Student s: studentsList){
    System.out.println(s);
}
```

Clase invelitoare

Tip primitiv	Clasa învelitoare
boolean	Boolean
char	Character
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double

```
Integer intObject1 = new Integer(34);
Integer intObject2 = new Integer("35");

Boolean boolValue1 = new Boolean("true");

Integer intObject3 = Integer.valueOf("36");
Integer intObject4 = Integer.valueOf("1001", 2); // 9 in baza
2

Integer intObj1 = 23;     //autoboxing
int intPrimitive = intObj1;    //unboxing
intPrimitive++;
Integer intObj2 = intPrimitive;    //autoboxing
```

Modificatorul static

Variabile statice - valori memorate la nivelul clasei si nu la nivelul fiecarei instante

Exemplu : declararea eficienta a constantelor

static final double PI =3.142

Metode statice - metode aplicabile la nivel de classa si nu de instanta

Se pot apela si doar cu numele clasei, nu este necesar sa cream o instanta

NumeClasa.numeMetodaStatica(listaParametri)

Utilizarea membrilor statici

```
public class Exemplu {
    int x ; // Variabila de instanta
    static long n; // Variabila de clasa
    public void metodaDeInstanta() {
        n ++; // Corect
        x --; // Corect
    }
    public static void metodaStatica() {
        n ++; // Corect
        x --; // Eroare la compilare !
    }
}
```

```
Exemplu.metodaStatica(); // Corect
Exemplu obj = new Exemplu();
obj.metodaStatica(); // Corect
```

```
Exemplu.metodaDeInstanta(); // Eroare
Exemplu obj = new Exemplu();
obj.metodaDeInstanta(); // Corect
```

Mostenirea

Este o relatie intre clase prin care o clasa (superclasa/clasa de baza) pune la dispozitia altor clase (subclase/clase derivate) structura si comportamentul definite de ea. O clasa poate extinde direct cel mult o clasa, nu exista mostenire multipla de clasa in Java.

```
class SuperClasa {
...
}
class SubClasa extends SuperClasa {
...
}
```

Constructorii in contextul mostenirii

In contextul mostenirii, cel in masura sa initializeze campurile mostenite de o clasa de baza este constructorul superclasei.

```
public class Student extends Persoana {
    private static final int COD_UNIVERSITATE = 15;
    private int anStudiu;

    public Student(String nume, byte varsta, int anStudiu)
    {
        super(nume,varsta); //apelconstr cu param din clasa de baza
        this.anStudiu=anStudiu;
    }
```

```
public class Persoana{
    protected String nume;
    protected byte varsta;
    public Persoana(){
        this("",(byte)0);
    }
    public Persoana(String nume, byte varsta){
        this.nume=nume;
        this.varsta=varsta;
    }
```

Daca in constructorul clasei Student prima instructiune nu ar fi fost **super**(nume, varsta) atunci s-ar fi apelat automat constructorul default Persoana( ). Daca acesta nu exista, primeam eroare/

Super in contextul mostenirii

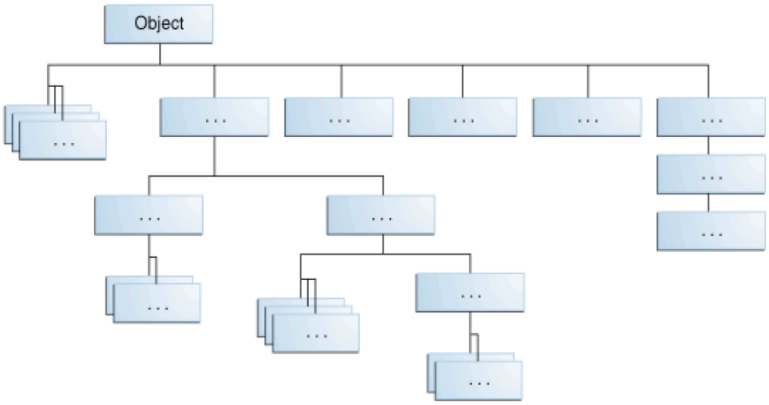
super se foloseste pentru :

1. A apela din constructorul clasei derivate un constructor al clasei de baza, apelul acelui constructor fiind prima instructiune din constructorul clasei derivate.
2. Pentru a referi un membru al clasei de baza in clasa derivata.

```
public class Persoana{
    protected String nume;
    protected byte varsta;

    public Persoana(String nume, byte varsta){
        this.nume=nume;
        this.varsta=varsta;
    }
    public void socializeaza() {
        System.out.println(nume + " socializeaza  ... ");
    }


public class Student extends Persoana {
    …
    public Student(String nume, byte varsta, int anStudiu)
    {
            super(nume,varsta); //apel constr cu param din clasa de baza
            this.anStudiu=anStudiu;
    }

    public void seDistreaza(){
        super.socializeaza(); //apel metoda din clasa de baza
        System.out.println("Si canta ....");
    }
```

Object - Superclasa a tuturor claselor

In Java orice clasa este derivata din Object



Metodele clasei Object

Toate obiectele, mostenesc metodele acestei clase:

- **toString** : returneaza reprezentarea ca sir de caractere a unui obiect
- **equals** : testeaza egalitatea continutului a doua obiecte
- **hashCode** : returneaza valoarea hash corespunzatoare unui obiect
- **getClass** : returneaza clasa din care a fost instantiat obiectul
- **clone** : creeaza o copie a obiectului (implicit : shallow copy)
- **finalize** : apelata de GC inainte de distrugerea obiectului
- **...**

Redefinirea (overriding) unei metode

Doar metodele de instanta se pot redefini

Exemplu : redefinirea in clasa Persoana a metodei toString() din clasa Object

```
@Override
public String toString() {
    return "nume='" + nume + '\'' + ", varsta=" + varsta ;
}
```

Legare dinamica

In Java, comportamentul de apel a metodelor este dinamic, adica decizia carei implementari a metodei sa fie apelata se realizeaza la runtime in functie de tipul real al obiectului.
Exemplu :

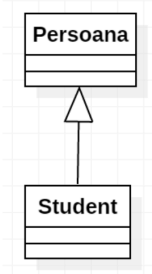
```
class A {
    void f( ) {
        System.out.println("A: metoda f");
    }
}

class B extends A {
    @Override
    void f( ) {
        System.out.println("B: metoda f");
    }
}

public class Main {
    public static void main( ) {
        A obj = new B( );
        obj.f( ); //B: metoda f
    }
}
```

Overriding vs overloading

Overriding - acelasi nume, aceeasi semnatura
Overloading - acelasi nume, semnaturi diferite

Upcasting vs downcasting

```
Persoana persoana;
Student student=new Student();
persoana=student;   //upcasting  (automat)


Student s2=(Student)persoana; //downcasting
explicit
```



Modificatori de acces in contextul mostenirii

- **private** - respectivul membru al clasei (atribut/metoda) poate fi accesat doar in interiorul clasei
- **public** - respectivul membru al clasei (atribut/metoda) poate fi accesat de oriunde
- **protected** - respectivul membru al clasei (atribut/metoda) poate fi accesat din interiorul clasei si din subclasele sale

Modificatorul final

Metodele final nu mai pot fi redefinite (overriding) in clasele derivate
Clasele final nu mai pot fi exstinse (e frunza in arborele ierarhiei de clase)

- final class A{}
- class B extends A{} - nu e posibil

Atributele, variabilele locale, argumentele final sunt constante

- constantele static final trebuie initializate la declarare

Clase abstracte

- Definesc concepte abstracte : forma, animal, figura...
- O metoda se numeste abstracta daca se declara, dar nu se defineste in clasa in care a fost declarata. O metoda abstracta se declara folosind cuvantul **abstract**.
  [modificator_acces] **abstract** tipReturnat nume([lista_param_formali]);
- O clasa abstracta este o clasa ce poate sa contina (nu este obligatoriu) metode abstracte.
- O clasa abstracta nu poate fi instantiata. ~~new~~
- Daca o clasa mosteneste o clasa abstracta si nu defineste toate metodele abstracte ale clasei de baza, trebuie declarata si ea abstracta.

Exemplu :

```
public abstract class Task {
    private String taskId;
    private String desc;

    public Task(String taskId, String desc) {
        this.taskId = taskId;
        this.desc = desc;
        System.out.println("Creating a task...");
    }

   public abstract void execute();
}
```

```
public class MessageTask extends Task {
    public String message;
    public MessageTask(String taskId, String desc, String message) {
        super(taskId, desc);
        this.message=message;
    }

    @Override
      public void execute() {
            System.out.println(message);
      }
}
```

Interfete

- O interfata in Java este un tip special de clasa abstracta care defineste doar comportamente (metode), dar nu le implementeaza.
- O interfata stabileste un contract, adica spune ce trebuie sa faca o clasa, nu cum trebuie sa o faca.
- Metodele **default** dintr-o interfata sunt singurele care permit implementare.
- Toate metodele declarate intr-o interfata sunt implicit publice.
- O interfata poate mosteni alta interfata si poate adauga metode noi.

Definirea unei interfete

```
public interface Formula {
    double PI=3.14; // este implicit public, static, final
    double calculate(double a, double b); //metoda abstracta

    default double sqrt(double a) {
        return Math.sqrt(a);
    }

    default double power(double a, double  b) {
        return Math.pow(a, b);
    }

    default double numarLaPatrat(double nr)
    {
        return power(nr,2);
    }

    default double numarLaCub(double nr)
    {
        return power(nr,3);
    }
}
```

Implementarea unei interfete

```
public interface Comparable{
    int compareTo(Object o);
}
public class Rational implements Comparable{
    private int numarator, numitor;
    //...
    public int compareTo(Object o){
        //...
    }
}
```

Clasa care implementeaza interfata este obligata sa defineasca toate metodele abstracte din interfata.
Daca cel putin o metoda abstracta din interfata nu este definita in clasa, atunci clasa trebuie declarata abstracta.